

IP TABLES

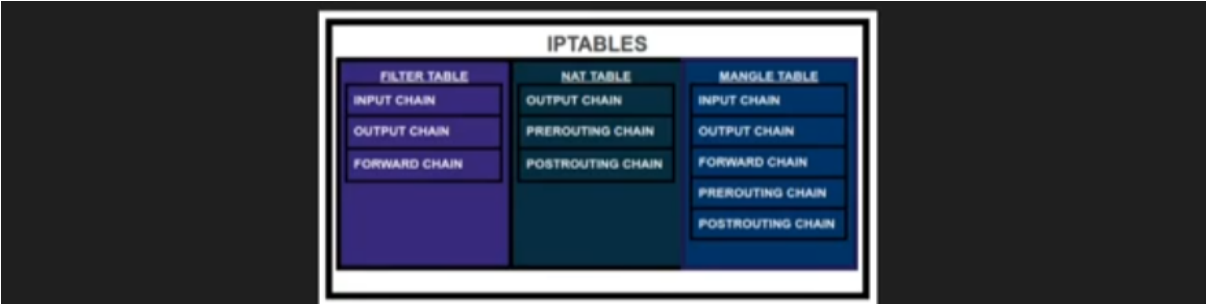
- Ip tables vs netfilter The simplest way to distinguish them is by their location and function: **Netfilter** is the **kernel framework**. It is the internal engine that actually intercepts, modifies, and redirects network packets as they move through the Linux networking stack. **Iptables** is the **user-space utility**. It is the command-line tool you use to write rules (like "block port 80") that tell Netfilter what to do.

Table	Primary Job	Real-World Example
Filter	Filtering / Security	"Block all incoming traffic on Port 22 except from my IP."
NAT	Address Translation	"Take traffic hitting my public IP on Port 80 and send it to internal Server A."
Mangle	Packet Modification	"Prioritize VOIP traffic over a slow VPN connection."

Chain	Filter Table	NAT Table	Mangle Table
PREROUTING		✓ (DNAT)	✓
INPUT	✓		✓
FORWARD	✓		✓
OUTPUT	✓	✓ (DNAT)	✓
POSTROUTING		✓ (SNAT)	✓

Beyond these five, you can create your **own chains**.

- **Why?** For organization. If you have 50 rules for a specific web application, you can create a chain called **WEB_RULES**, jump to it from the **INPUT** chain, and keep your configuration clean and modular.



A chain is literally a "chain of rules." When a packet hits a checkpoint, the system looks at the rules in that chain one by one (from top to bottom) until it finds a match.

Every packet consists of three main parts:

1. The Header (The "Envelope")

This is the metadata that tells the network where the packet is going and how to handle it. It contains:

- **Source IP Address:** Where it came from.
- **Destination IP Address:** Where it's going.
- **Protocol:** (e.g., TCP or UDP) How the data should be handled.
- **Packet Number:** (e.g., "Part 5 of 100") so the receiving computer knows how to reassemble them in order.

2. The Payload (The "Letter")

This is the actual data being transmitted. It could be a tiny snippet of a high-resolution photo, a few characters of a text message, or a millisecond of audio from a song.

3. The Trailer (The "Seal")

Also called the "footer," this usually contains a **checksum**. This is a bit of math that the receiving computer uses to verify the packet wasn't corrupted or changed during its trip across the wires.

Packets don't all follow the same path. Because the internet is a vast web of routers, Packet A might go through a server in New York, while Packet B (from the same file) might go through a server in Chicago because it's less congested at that exact microsecond.

1. **Fragmentation:** Your computer chops the file into packets.
2. **Routing:** Routers read the **Header** and pass the packet to the next "hop."
3. **Reassembly:** Your friend's computer receives all the packets, looks at the **Packet Numbers**, and puts them back together to show the original file.

Inside the filter table, there are three primary paths (chains) a packet can take. The kernel decides which path to use based on the packet's **destination**.

- **INPUT Chain:** For packets arriving at your system and destined for a local process (like your web server, an SSH session, or a database).
- **OUTPUT Chain:** For packets created by your system that are trying to go out to the internet (like a software update request or an email you sent).
- **FORWARD Chain:** For packets that arrive at your system but are meant for *somewhere else*. Your system acts as a "middleman" or router.

The "Targets" (The Verdict)

Once a packet matches a rule in the filter table, it is given a "Target" or verdict:

ACCEPT: Let the packet through.

- **DROP:** Forget the packet ever existed (don't even send a "rejected" message back). The sender will just "time out."
- **REJECT:** Block the packet and send a "Destination Unreachable" message back to the sender.
- **LOG:** Don't block it, just write down that it passed by (usually in `/var/log/kern.log`).

Action	Which Chain?	Where is the packet going?
Blocking a Hacker	INPUT	Into your computer.
Limiting Outbound Traffic	OUTPUT	Leaving your computer.
Setting up a VPN/Router	FORWARD	Passing through your computer.

Man iptables (to view full documentation)

```
iptables(8)          iptables 1.8.7          IPTABLES(8)
NAME
    iptables/ip6tables - administration tool for IPv4/IPv6 packet filtering and NAT

SYNOPSIS
    iptables [-t table] [-A|-C|-D] chain rule-specification
    ip6tables [-t table] [-A|-C|-D] chain rule-specification
    iptables [-t table] -I chain [rulenum] rule-specification
    iptables [-t table] -R chain rulenum rule-specification
    iptables [-t table] -D chain rulenum
    iptables [-t table] -S [chain [rulenum]]
    iptables [-t table] [-F|-L|-Z] [chain [rulenum]] [options...]
    iptables [-t table] -N chain
    iptables [-t table] -X [chain]
    iptables [-t table] -P chain target
    iptables [-t table] -E old-chain-name new-chain-name
    rule-specification = [matches...] [target]
    match = -m matchname [per-match-options]
    target = -j targetname [per-target-options]

DESCRIPTION
    Iptables and ip6tables are used to set up, maintain, and inspect the tables of IPv4 and IPv6 packet filter rules in the Linux kernel. Several different tables may be defined. Each table contains a number of built-in chains and may also contain user-defined chains.

Manual page iptables(8) line 1 (press h for help or q to quit)
```

Each chain is a list of rules which can match a set of packets. Each rule specifies what to do with a packet that matches. This is called a 'target', which may be a jump to a user-defined chain in the same table.

TARGETS

A firewall rule specifies criteria for a packet and a target. If the packet does not match, the next rule in the chain is examined; if it does match, then the next rule is specified by the value of the target, which can be the name of a user-defined chain, one of the targets described in `iptables-extensions(8)`, or one of the special values `ACCEPT`, `DROP` or `RETURN`.

`ACCEPT` means to let the packet through. `DROP` means to drop the packet on the floor. `RETURN` means stop traversing this chain and resume at the next rule in the previous (calling) chain. If the end of a built-in chain is reached or a rule in a built-in chain with target `RETURN` is matched, the target specified by the chain policy determines the fate of the packet.

TABLES

There are currently five independent tables (which tables are present at any time depends on the kernel configuration options and which modules are present).

`-t, --table table`

This option specifies the packet matching table which the command should operate on. If the kernel is configured with automatic module loading, an attempt will be made to load the appropriate module for that table if it is not already there.

The tables are as follows:

filter:

This is the default table (if no `-t` option is passed). It contains the built-in chains `INPUT` (for packets destined to local sockets), `FORWARD` (for packets being routed through the box), and `OUTPUT` (for locally-generated packets).

```

routed through the box), and OUTPUT (for locally-generated packets).

nat:
This table is consulted when a packet that creates a new connection is encountered. It consists of four built-ins: PREROUTING (for altering packets as soon as they come in), INPUT (for altering packets destined for local sockets), OUTPUT (for altering locally-generated packets before routing), and POSTROUTING (for altering packets as they are about to go out). IPv6 NAT support is available since kernel 3.7.

mangle:
This table is used for specialized packet alteration. Until kernel 2.4.17 it had two built-in chains: PREROUTING (for altering incoming packets before routing) and OUTPUT (for altering locally-generated packets before routing). Since kernel 2.4.18, three other built-in chains are also supported: INPUT (for packets coming into the box itself), FORWARD (for altering packets being routed through the box), and POSTROUTING (for altering packets as they are about to go out).

raw:
This table is used mainly for configuring exemptions from connection tracking in combination with the NOTRACK target. It registers at the netfilter hooks with higher priority and is thus called before ip_conntrack, or any other IP tables. It provides the following built-in chains: PREROUTING (for packets arriving via any network interface) OUTPUT (for packets generated by local processes)

```

sudo iptables -L : To view default policies

Iptables –policy INPUT ACCEPT ; changing default policy

1. -A, --append : Append to the chain provided in parameters.

Syntax

```
iptables [-t table] --append [chain] [parameters]
```

Example:

This command drops all the traffic coming on any port.

```
iptables -t filter --append INPUT -j DROP
```

2. -D, --delete : Delete rule from the specified chain.

Syntax

```
iptables [-t table] --delete [chain] [rule_number]
```

Example:

This command deletes the rule 2 from INPUT chain.

```
iptables -t filter --delete INPUT 2
```

<https://www.geeksforgeeks.org/linux-unix/iptables-command-in-linux-with-examples/>
for more